



REACT.JS

Interview Questions & Answers

College Coders Edition • All 26 Phases • Fresher & Job-Ready

☐ COVERAGE

- ✓ React Fundamentals, JSX, Props, State, Events, Forms
- ✓ useEffect, Lists, Styling, useRef, Context API
- ✓ API Handling, React Router (all patterns), Local Storage
- ✓ Performance: memo, useMemo, useCallback, Code Splitting

☐ DIFFICULTY LEGEND ☐ BASIC ☐ PRACTICAL ☐ ADVANCED

PHASE 1 — React Fundamentals

Q1. What is React? ☐ BASIC

Short Answer: React is an open-source JavaScript library built by Facebook for creating fast and interactive user interfaces.

Detailed Explanation: React focuses only on the View layer of an application (the 'V' in MVC). It lets developers build reusable UI components and manage how the UI updates efficiently when data changes. React was released in 2013 and is maintained by Meta (Facebook) and a large open-source community.

Practical Example: Instead of manually updating the DOM when a user likes a post, React automatically re-renders only the 'Like' button component when its state changes.

☐ **Interview Tip:** Always mention 'component-based' and 'declarative UI' in your answer — interviewers love these key terms.

Q2. What are the key features of React? ☐ BASIC

Short Answer: Key features: JSX, Virtual DOM, Component-Based Architecture, One-Way Data Binding, and React Hooks.

Detailed Explanation: 1) JSX – lets you write HTML inside JavaScript. 2) Virtual DOM – React creates a lightweight copy of the real DOM and updates only changed parts. 3) Component-Based – UI is broken into independent, reusable pieces. 4) One-Way Data Flow – data flows from parent to child via props, making debugging easier. 5) Hooks – functions like `useState` and `useEffect` give functional components state and lifecycle capabilities.

Practical Example: A Navbar, Sidebar, and Footer on a website are each separate React components that can be reused across multiple pages.

□ **Interview Tip:** Memorise at least 5 features and be ready to explain each one briefly.

Q3. What is the Virtual DOM? How is it different from the Real DOM? □ BASIC

Short Answer: Virtual DOM is a lightweight JavaScript copy of the real DOM. React uses it to compute minimal UI changes before touching the browser's real DOM.

Detailed Explanation: When state or props change, React creates a new Virtual DOM tree and compares it to the previous one using a process called 'diffing'. It then applies only the differences (patches) to the real DOM. This avoids expensive full-page re-renders. The Real DOM is slow to update because every change triggers layout recalculations, repaints, and reflows in the browser.

Practical Example: In a 1000-item list, if only item #5 changes, React updates only that one node in the real DOM instead of re-rendering all 1000 items.

□ **Interview Tip:** Draw a simple diagram in your mind: Virtual DOM → diffing → patching Real DOM. Explain this flow clearly.

Q4. Why is React fast? □ BASIC

Short Answer: React is fast because of the Virtual DOM, efficient diffing algorithm, and component-level re-rendering.

Detailed Explanation: React batches multiple state updates together to reduce DOM operations. It uses a reconciliation algorithm to identify the minimum number of changes required. Additionally, with `React.memo`, `useMemo`, and `useCallback`, developers can prevent unnecessary re-renders. Code splitting and lazy loading further improve initial load performance.

Practical Example: Without React, changing a user's profile photo might re-render the entire page. React only re-renders the Avatar component.

□ **Interview Tip:** Mention 'reconciliation algorithm' and 'batching' — these show depth of knowledge.

Q5. What is a Single Page Application (SPA)? □ BASIC

Short Answer: An SPA is a web app that loads a single HTML page and dynamically updates content without full page reloads.

Detailed Explanation: In a traditional multi-page app, navigating to a new URL fetches a completely new HTML page from the server. In an SPA, the initial HTML is loaded once, and JavaScript handles routing and rendering new views. React combined with React Router enables SPA behaviour. This makes apps feel faster and more native-like.

Practical Example: Gmail, Google Maps, and Facebook are SPAs — when you click inbox vs sent, the page doesn't reload; only the content area updates.

□ **Interview Tip:** Interviewers often follow up with: 'What are the drawbacks of SPAs?' — mention SEO challenges and initial load time.

Q6. What is the difference between Vite and Create React App (CRA)? □ PRACTICAL

Short Answer: Vite is a modern, fast build tool using ES Modules. CRA is the traditional React setup tool that uses Webpack under the hood.

Detailed Explanation: CRA bundles all files at startup, making it slow for large projects. Vite uses native ES Modules in development, serving only the files the browser requests, making it significantly faster. Vite's hot module replacement (HMR) is near-instant. CRA is being deprecated and the React team now recommends Vite or Next.js for new projects.

Practical Example: A Vite project starts a dev server in under 1 second. A CRA project might take 10–30 seconds for large apps.

□ **Interview Tip:** Mention that CRA is deprecated. Saying 'I prefer Vite because of faster HMR' shows you are updated.

Q7. Explain the standard folder structure of a React project. □ BASIC

Short Answer: A React project has src/ (components, pages, hooks, context), public/ (static assets), and config files like package.json and vite.config.js.

Detailed Explanation: The src/ folder contains: components/ (reusable UI), pages/ (route-level components), hooks/ (custom hooks), context/ (global state), assets/ (images, fonts), and utils/ (helper functions). The public/ folder holds static files served directly. package.json lists dependencies. A clean folder structure improves maintainability and team collaboration.

Code Example:

```
src/  
  components/  
  pages/  
  hooks/  
  context/  
public/  
package.json  
vite.config.js
```

□ **Interview Tip:** Describe this from memory in interviews. It shows project experience even if you're a fresher.

Q8. What is a Component in React? □ BASIC

Short Answer: A component is an independent, reusable piece of UI that can accept inputs (props) and return JSX to render on screen.

Detailed Explanation: React components are the building blocks of any React application. Each component manages its own logic and appearance. Components can be composed together — a Page component might contain Header, Sidebar, and Footer components. There are two types: Functional Components (modern, recommended) and Class Components (legacy).

Practical Example: A Button component can be written once and used on every page with different labels and colours by passing different props.

□ **Interview Tip:** Emphasise 'reusable' and 'independent' — these are the core strengths of component architecture.

Q9. What is a Functional Component? □ BASIC

Short Answer: A functional component is a plain JavaScript function that returns JSX. It is the modern, recommended way to write React components.

Detailed Explanation: Before React 16.8, functional components couldn't manage state or use lifecycle methods. With the introduction of Hooks (useState, useEffect, etc.), functional components are now fully capable. They are simpler, shorter, and easier to test than class components.

Code Example:

```
function Welcome({ name }) {  
  return <h1>Hello, {name}!</h1>;  
}
```

□ **Interview Tip:** Always write functional components in interviews unless the question specifically asks about class components.

Q10. What is rendering and re-rendering in React? □ PRACTICAL

Short Answer: Rendering is when React calls a component function to produce JSX output. Re-rendering happens when state or props change.

Detailed Explanation: On initial render, React builds the component tree and commits it to the DOM. A re-render is triggered when: 1) a component's state changes, 2) its parent re-renders, or 3) a context it consumes changes. React compares the new output with the previous Virtual DOM and only updates the real DOM where differences exist. Unnecessary re-renders should be avoided for performance.

Practical Example: When you type in an input field with onChange updating state, the component re-renders on every keystroke.

□ **Interview Tip:** Interviewers often ask 'what causes a re-render?' — know all 3 reasons: state change, parent re-render, context change.

PHASE 2 — JSX

Q11. What is JSX? □ BASIC

Short Answer: JSX (JavaScript XML) is a syntax extension for JavaScript that lets you write HTML-like code inside JavaScript files.

Detailed Explanation: JSX is not valid JavaScript — it gets compiled to React.createElement() calls by Babel before running in the browser. JSX makes component code more readable and visually similar to the HTML output it produces. Every JSX element becomes a function call that returns a React element object.

Code Example:

```
// JSX
const el = <h1>Hello</h1>;
// Compiled to:
const el = React.createElement('h1', null, 'Hello');
```

□ **Interview Tip:** Always clarify: 'JSX is syntactic sugar for React.createElement()' — this impresses interviewers.

Q12. How do you write JavaScript inside JSX? □ BASIC

Short Answer: You use curly braces {} to embed any valid JavaScript expression inside JSX.

Detailed Explanation: Inside {}, you can write variables, function calls, arithmetic, ternary operators, and array methods like .map(). You cannot write statements (if, for loops) directly — only expressions are allowed inside curly braces.

Code Example:

```
const name = 'Priya';
return <h1>Hello, {name.toUpperCase()}!</h1>;
```

□ **Interview Tip:** Remember: {} takes expressions, not statements. Replace if/else with ternary inside JSX.

Q13. What are the important rules of JSX? □ BASIC

Short Answer: JSX rules: return one root element, close all tags, use className instead of class, and use camelCase for attributes.

Detailed Explanation: 1) Every component must return a single root element (use a Fragment `<></>` to avoid extra DOM nodes). 2) All tags must be self-closed if empty: `<img />`, `<input />`. 3) Use className instead of class, htmlFor instead of for. 4) Event attributes use camelCase: onClick, onChange, onSubmit. 5) JavaScript expressions go inside `{}`.

Code Example:

```
// Wrong
return <h1>Hi</h1><p>World</p>;
// Correct
return <><h1>Hi</h1><p>World</p></>;
```

□ **Interview Tip:** Mention React Fragment (`<></>`) — it avoids unnecessary wrapper divs and shows you know best practices.

Q14. What is a React Fragment and why is it used? □ PRACTICAL

Short Answer: A Fragment is an invisible wrapper that lets you return multiple JSX elements without adding an extra HTML node to the DOM.

Detailed Explanation: JSX requires a single root element. If you wrap in a `<div>`, it adds an extra node to the DOM which can break CSS layouts (like flexbox or grid). Fragments (`<></>` or `<React.Fragment>`) solve this cleanly without polluting the DOM structure.

Code Example:

```
return (
  <>
    <Header />
    <Main />
    <Footer />
  </>
);
```

□ **Interview Tip:** Explain Fragment with a CSS layout problem scenario — it shows real-world thinking.

PHASE 3 — Props

Q15. What are Props in React? □ BASIC

Short Answer: Props (short for properties) are read-only inputs passed from a parent component to a child component.

Detailed Explanation: Props allow components to be dynamic and reusable. A parent passes data to a child by adding attributes to the JSX element. The child receives props as a parameter in its function. Props can be strings, numbers, booleans, arrays, objects, or even functions.

Code Example:

```
// Parent
<UserCard name="Arjun" age={22} />
// Child
function UserCard({ name, age }) {
  return <p>{name} is {age} years old</p>;
}
```

```
}
```

□ **Interview Tip:** Immediately mention 'Props are read-only' — it's a very common follow-up question.

Q16. Why are Props read-only? □ BASIC

Short Answer: Props are read-only to maintain unidirectional data flow and prevent child components from accidentally modifying parent data.

Detailed Explanation: React follows a one-way data flow pattern. If a child could modify props, it would make debugging very hard — you wouldn't know which component changed the data. This principle is called 'immutability of props'. If a child needs to communicate back to the parent, it should call a callback function passed as a prop.

Practical Example: If you receive your salary slip (props), you can read it but you cannot modify your employer's payroll system from your side.

□ **Interview Tip:** This is a favourite interview question. The answer is: 'To maintain unidirectional data flow and predictable component behaviour.'

Q17. What is props destructuring and why is it used? □ BASIC

Short Answer: Destructuring extracts specific values from the props object directly in the function parameter, making code cleaner.

Detailed Explanation: Without destructuring, you'd write props.name, props.age everywhere. With destructuring, you extract them in the function signature and use them directly. This reduces repetition and improves code readability.

Code Example:

```
// Without destructuring
function Card(props) { return <h1>{props.title}</h1>; }
// With destructuring
function Card({ title, desc }) { return <h1>{title}</h1>; }
```

□ **Interview Tip:** Always use destructuring in interviews — it demonstrates clean coding practices.

Q18. What is the difference between Props and State? □ PRACTICAL

Short Answer: Props are external inputs passed from parent to child (read-only). State is internal data managed by the component itself (mutable).

Detailed Explanation: Props: passed from outside, cannot be changed by the component, used for parent-to-child communication. State: defined inside the component using useState(), can be updated, triggers re-render when updated. Think of props as a component's configuration and state as its memory.

Practical Example: A theme='dark' prop tells a component what to display. A count state tracks how many times a button was clicked.

□ **Interview Tip:** Draw a mental table: Props = external, read-only, parent-controlled. State = internal, mutable, self-controlled.

PHASE 4 — State & useState

Q19. What is State in React? ▢ BASIC

Short Answer: State is a built-in object in a component that holds dynamic data. When state changes, React automatically re-renders the component.

Detailed Explanation: State is the component's internal memory. Unlike props, state can be changed using a setter function. Every time `setState` is called, React schedules a re-render and updates the UI to reflect the new state. State management is fundamental to building interactive applications.

Practical Example: In a shopping cart, the list of added items is stored in state. When you add or remove an item, the state updates and the cart UI re-renders.

▢ **Interview Tip:** Key phrase: 'State drives UI — when state changes, UI updates automatically.' Always use this in your answer.

Q20. What is the `useState()` Hook and how does it work? ▢ BASIC

Short Answer: `useState()` is a React Hook that adds state to functional components. It returns the current state value and a function to update it.

Detailed Explanation: `useState(initialValue)` returns an array with two elements: the current state and the setter function. When the setter is called with a new value, React re-renders the component with the updated state. The initial value can be any data type: string, number, boolean, array, or object.

Code Example:

```
const [count, setCount] = useState(0);  
// Update state:  
setCount(count + 1);
```

▢ **Interview Tip:** Always explain: 'useState returns a pair — the value and the updater function.' Pronounce it clearly.

Q21. Why do we use `const` with `useState` even though state changes? ▢ ADVANCED

Short Answer: `const` is safe here because React re-renders the component (calls the function again), creating a new `const` binding with the updated value each time.

Detailed Explanation: The variable itself (`count`) never changes within a single render — it's the result of `useState()` at that render. When `setCount` is called, React calls the component function again, creating a completely new execution context where `count` is reassigned via `const` with the new value. The old render and its `const` variable simply go out of scope.

Practical Example: Think of it like a snapshot: each render is a photograph with a fixed count. A new photo (render) has a new count value.

▢ **Interview Tip:** This is a popular tricky question. The answer shows you understand how React's rendering model works at a conceptual level.

Q22. Why is state update asynchronous in React? ▢ ADVANCED

Short Answer: React batches multiple state updates together and applies them after the current event handler completes, to avoid unnecessary re-renders.

Detailed Explanation: When you call `setState`, React doesn't immediately update the state — it queues the update. Multiple `setState` calls in the same event are batched and processed together in one re-render. This is called 'batching'. If you need state to depend on the previous state, use the functional form: `setState(prev => prev + 1)`.

Code Example:

```
// WRONG - may not reflect latest state:  
setCount(count + 1);  
setCount(count + 1); // Both use same 'count'  
// CORRECT:
```

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1); // Uses updated prev each time
```

□ **Interview Tip:** Always use the functional form 'prev => prev + 1' when the new state depends on the old state. Mention batching.

Q23. What is 'Lifting State Up' in React? □ PRACTICAL

Short Answer: Lifting state up means moving state from a child component to the nearest common parent so multiple sibling components can share it.

Detailed Explanation: When two sibling components need to share or sync data, you lift the state to their common parent. The parent then passes the state down as props and passes update functions as callback props. This is the React-approved pattern before introducing Context API or state management libraries.

Practical Example: A temperature converter with Celsius and Fahrenheit inputs — both inputs need to stay in sync, so the temperature state is lifted to their parent component.

□ **Interview Tip:** Lifting state is a concept that also leads to props drilling — mention that Context API or Redux solve this at scale.

PHASE 5 — Conditional Rendering

Q24. What are the different ways to do conditional rendering in React? □ BASIC

Short Answer: Three main ways: if/else outside JSX, ternary operator (?:) inside JSX, and logical AND (&&) for show-or-nothing cases.

Detailed Explanation: 1) if/else: Use before the return statement to decide what JSX to return. 2) Ternary (condition ? A : B): Use inside JSX for two alternate outputs. 3) Logical && (condition &&<Component/>): Renders the component only when the condition is true, renders nothing when false. Each approach is suited to different scenarios.

Code Example:

```
// Ternary  
{isLoggedIn ? <Dashboard /> : <Login />}  
// Logical &&  
{hasError &&<ErrorMessage />}
```

□ **Interview Tip:** Know all three methods. Interviewers often ask which is best for which case — '&&' for show/hide, ternary for A-or-B.

Q25. What is the risk of using && for conditional rendering? □ ADVANCED

Short Answer: If the condition is 0 or NaN (falsy numbers), React renders them literally on screen instead of rendering nothing.

Detailed Explanation: In JavaScript, 0 &&<Component/> returns 0 (not false). React renders 0 as text on screen. To avoid this, always convert the condition to a boolean using !! or explicit comparison: {count > 0 &&<List />} instead of {count &&<List />}.

Code Example:

```
// BUG: renders '0' when count is 0  
{count &&<ItemList />}  
// FIXED:
```



```
{count > 0 &&<ItemList />}
```

❑ **Interview Tip:** This is a classic gotcha question. Saying this in an interview immediately marks you as an experienced developer.

PHASE 6 — Event Handling

Q26. How does event handling work in React? ❑ BASIC

Short Answer: React uses camelCase synthetic event attributes like `onClick`, `onChange`, and `onSubmit`. You pass a function reference (not a call) as the event handler.

Detailed Explanation: React wraps browser events in a `SyntheticEvent` — a cross-browser wrapper that normalises event behaviour. Handlers receive this event object as a parameter. Important: pass the function reference (`onClick={handleClick}`) not a call (`onClick={handleClick()}`) — calling it immediately executes on render.

Code Example:

```
// Correct:
<button onClick={handleClick}>Click</button>
// Wrong (executes on render):
<button onClick={handleClick()}>Click</button>
```

❑ **Interview Tip:** The difference between `onClick={fn}` and `onClick={fn()}` is a very popular interview question. Know this cold.

Q27. What is `preventDefault()` and when is it used? ❑ PRACTICAL

Short Answer: `preventDefault()` stops the browser's default behaviour for an event, like preventing form submission from reloading the page.

Detailed Explanation: When a form is submitted, the browser's default action is to reload the page and send a GET/POST request. In React SPAs, you want to handle form submission in JavaScript without a page reload. Calling `e.preventDefault()` in the `onSubmit` handler prevents this default behaviour.

Code Example:

```
function handleSubmit(e) {
  e.preventDefault(); // Prevent page reload
  console.log('Form submitted:', formData);
}
```

❑ **Interview Tip:** Always mention forms and page reload prevention. Also mention it works with links to prevent navigation.

Q28. How do you pass arguments to an event handler? ❑ PRACTICAL

Short Answer: Wrap the handler in an arrow function: `onClick={() => handleClick(id)}` so it's called only when the event fires, not on render.

Detailed Explanation: You can't write `onClick={handleDelete(id)}` because that executes immediately. The solution is to wrap it: `onClick={() => handleDelete(id)}`. The arrow function is a new function that, when clicked, calls `handleDelete(id)`. This is the standard React pattern for passing arguments.

Code Example:

```
// Correct way to pass arguments:
```

```
<button onClick={() => handleDelete(item.id)}>Delete</button>
```

□ **Interview Tip:** This is asked in almost every React fresher interview. Practice explaining the 'why' behind the arrow function wrapper.

Q29. What is e.target.value and where is it used? □ BASIC

Short Answer: e.target.value is the current value of a form input element captured from the change event object.

Detailed Explanation: When a user types in an input, the onChange event fires. The event object 'e' contains 'target' — the DOM element — and 'value' — the current text in that input. This is the primary way to read user input in React controlled components.

Code Example:

```
function handleChange(e) {  
  setName(e.target.value);  
}  
  
<input onChange={handleChange} />
```

□ **Interview Tip:** Know this by heart. You'll use e.target.value in almost every form you build in React.

PHASE 7 — Forms

Q30. What is a Controlled Component in React? □ BASIC

Short Answer: A controlled component is a form element whose value is controlled by React state, making React the single source of truth.

Detailed Explanation: In a controlled component, the input's value is set by state, and onChange updates that state. Every keystroke updates state and the state value drives what the input shows. This gives React full control over the form data. An uncontrolled component, by contrast, stores its value in the DOM itself (using useRef).

Code Example:

```
const [email, setEmail] = useState('');  
  
<input  
  value={email}  
  onChange={(e) => setEmail(e.target.value)}  
>
```

□ **Interview Tip:** Always say 'single source of truth' when explaining controlled components — it's the key phrase interviewers want to hear.

Q31. How do you handle multiple form inputs with a single handler? □ PRACTICAL

Short Answer: Use a single object state and a reusable handleChange function that uses e.target.name to update the correct field dynamically.

Detailed Explanation: Instead of creating separate state and handler for each input, store all form data in one object. Use name attributes on inputs matching the state keys. The handler uses computed property names [e.target.name] to update the right field. This scales beautifully for large forms.

Code Example:

```
const [form, setForm] = useState({ name: '', email: '' });
```

```
function handleChange(e) {
  setForm({ ...form, [e.target.name]: e.target.value });
}
<input name='name' value={form.name} onChange={handleChange} />
<input name='email' value={form.email} onChange={handleChange} />
```

□ **Interview Tip:** This pattern is extremely common in real projects. Practise it until you can write it from memory.

PHASE 8 — useEffect

Q32. What is useEffect and what are side effects? □ BASIC

Short Answer: useEffect is a Hook for running side effects in functional components. Side effects are operations that interact with the outside world: API calls, timers, DOM updates, subscriptions.

Detailed Explanation: React components should be pure — same input = same output. But apps need to interact with external systems. useEffect is the designated place for these interactions. It runs after the component renders, keeping the render phase pure. Examples of side effects: fetching data, setting up event listeners, updating document title, starting timers.

Code Example:

```
useEffect(() => {
  document.title = 'New Title';
}, []); // Runs once after mount
```

□ **Interview Tip:** List at least 3 examples of side effects: API call, timer, document title. This shows breadth of knowledge.

Q33. Explain the dependency array in useEffect. □ BASIC

Short Answer: The dependency array controls when useEffect runs. No array = runs every render. Empty [] = runs once. [value] = runs when value changes.

Detailed Explanation: 1) No dependency array: useEffect runs after every single render. 2) Empty array []: runs only once after the initial mount — equivalent to componentDidMount. 3) Array with values [userId]: runs after mount AND whenever userId changes. This pattern is used to re-fetch data when a parameter changes.

Code Example:

```
useEffect(() => { fetchUser(id); }, [id]); // Re-runs when 'id' changes
```

□ **Interview Tip:** This is the most asked useEffect question. Know all three cases perfectly. Interviewers often ask what happens if you forget the dependency array.

Q34. What is the cleanup function in useEffect? □ PRACTICAL

Short Answer: A cleanup function is returned from useEffect to cancel subscriptions, clear timers, or remove event listeners when the component unmounts or before the effect re-runs.

Detailed Explanation: Without cleanup, effects like timers and subscriptions keep running even after a component is removed from the DOM — causing memory leaks. The cleanup function runs before the component unmounts and before the effect runs again on re-render. It's the equivalent of componentWillUnmount in class components.

Code Example:

```
useEffect(() => {
```

```
const timer = setInterval(() => tick(), 1000);
return () => clearInterval(timer); // Cleanup!
}, []);
```

▢ **Interview Tip:** Always mention 'memory leaks' when explaining cleanup functions — it demonstrates real-world awareness.

Q35. What happens if you put a `setState` call inside `useEffect` without a dependency array? ▢ **ADVANCED**

Short Answer: It causes an infinite loop: the component renders → `useEffect` runs → `setState` triggers re-render → `useEffect` runs again → forever.

Detailed Explanation: Without a dependency array, `useEffect` runs after every render. If it calls `setState`, that triggers a re-render, which triggers `useEffect` again, and so on. This is one of the most common React bugs for beginners. Always add a proper dependency array to control when the effect should run.

Practical Example: Like a phone notification that keeps alerting you every time you look at the notification itself.

▢ **Interview Tip:** This is a real bug scenario question. Knowing the cause (no dependency array + `setState`) and solution shows practical experience.

PHASE 9 — Lists & Keys

Q36. How do you render a list of items in React? ▢ **BASIC**

Short Answer: Use the JavaScript `.map()` method to transform an array of data into an array of JSX elements.

Detailed Explanation: `.map()` iterates over each item and returns a JSX element for it. React renders the resulting array as a list of elements. This is the primary, idiomatic way to render dynamic lists in React. Avoid using for loops inside JSX — use `.map()` instead.

Code Example:

```
const fruits = ['Apple', 'Mango', 'Banana'];
return (
  <ul>
    {fruits.map((fruit, i) => <li key={i}>{fruit}</li>)}
  </ul>
);
```

▢ **Interview Tip:** Never forget the `key` prop on list items — it's almost always a follow-up question.

Q37. What are Keys in React and why are they important? ▢ **BASIC**

Short Answer: Keys are unique identifiers added to list items to help React efficiently identify which items changed, were added, or removed.

Detailed Explanation: React uses keys during reconciliation to match up elements between renders. Without keys, React re-renders the entire list on any change. With stable, unique keys, React only updates the items that actually changed. Keys must be unique among siblings, stable across renders, and should not be the array index if the list order can change.

Code Example:

```
// Good: use unique id
```

```
{users.map(user =><UserCard key={user.id} user={user} />)}  
// Avoid: index as key for dynamic lists  
{users.map((user, i) =><UserCard key={i} user={user} />)}
```

□ **Interview Tip:** Explain why index keys are bad: 'If you reorder or delete items, indices shift and React gets confused, causing UI bugs.'

Q38. When is it acceptable to use index as a key? □ **ADVANCED**

Short Answer: Using index as key is acceptable only when the list is static (never reordered or filtered) and items have no unique IDs.

Detailed Explanation: Index keys cause bugs when the list is dynamic because React tracks elements by their key. If you delete item at index 1, item 2 becomes index 1, and React might re-use the wrong component state. For static display-only lists where items are never reordered, index keys are fine and harmless.

Practical Example: A static list of 3 tabs that never change order — using index as key is acceptable here.

□ **Interview Tip:** This shows senior-level awareness. The rule: always use ID from data if available; index only for completely static lists.

PHASE 10 — Conditional Styling

Q39. How do you apply conditional styles in React? □ **PRACTICAL**

Short Answer: Use inline style with a ternary for style objects, or use template literals / conditional className with CSS classes.

Detailed Explanation: Method 1: Inline style — apply a JavaScript style object conditionally using ternary. Method 2: className — dynamically build class names using template literals or libraries like clsx. Method 3: For complex cases, use CSS modules or styled-components. The className approach is preferred because it keeps styles in CSS files.

Code Example:

```
// Inline style  
<div style={{ color: isError ? 'red' : 'green' }}>Text</div>  
// className  
<button className={isActive ? 'btn-active' : 'btn-default'}>Click</button>
```

□ **Interview Tip:** Mention clsx or classnames library for complex conditional class combinations — it shows production-level experience.

PHASE 11 — useRef

Q40. What is useRef and when would you use it? □ **BASIC**

Short Answer: useRef creates a mutable ref object whose .current property persists across renders without causing re-renders when changed.

Detailed Explanation: useRef has two main uses: 1) DOM Access — attach to a JSX element via the ref attribute to directly interact with the DOM (focus, scroll, measure). 2) Storing mutable values — store values that should persist across renders but shouldn't trigger re-renders (like timer IDs, previous values).

Code Example:

```
const inputRef = useRef(null);  
// Auto-focus:  
useEffect(() => inputRef.current.focus(), []);  
<input ref={inputRef} />
```

□ **Interview Tip:** Key point: 'useRef does NOT trigger re-renders when .current changes' — this is what differentiates it from useState.

Q41. What is the difference between useRef and useState? □ PRACTICAL

Short Answer: useState triggers re-render on update; useRef does not. Use useState for UI values, useRef for values that shouldn't affect the UI.

Detailed Explanation: useState: changing the value causes the component to re-render. The new value is reflected in JSX. useRef: changing .current does NOT cause re-render. The value is stored but the UI doesn't react to it. Use useRef to store timer IDs, previous state values, or DOM element references where triggering a re-render would be wasteful.

Practical Example: Tracking how many times a component rendered: store count in useRef so you can access it without causing more renders.

□ **Interview Tip:** This is a frequent interview question. The one-line answer: 'useState is for UI state, useRef is for side-effect storage without re-rendering.'

PHASE 12–13 — Advanced State Flow & Props Drilling

Q42. What is Props Drilling and why is it a problem? □ BASIC

Short Answer: Props drilling is passing data through multiple layers of intermediate components that don't need it, just to reach a deeply nested child.

Detailed Explanation: In a large component tree, if the root component has data needed by a grandchild, you must pass it through every intermediate component as props. This makes intermediate components tightly coupled to data they don't use, making code hard to maintain and refactor. As the app grows, this becomes unmanageable.

Practical Example: App passes userId → Header → Navbar → ProfileIcon. Only ProfileIcon needs userId, but it had to pass through Header and Navbar.

□ **Interview Tip:** Always follow up with the solution: 'Context API or state management libraries like Redux solve props drilling.'

PHASE 14 — Context API

Q43. What is the Context API in React? ▢ BASIC

Short Answer: Context API is a built-in React feature that allows you to share data globally across the component tree without props drilling.

Detailed Explanation: Context provides a way to pass data through the component tree without manually passing props at every level. It consists of three parts: 1) `React.createContext()` — creates the context. 2) `Context.Provider` — wraps components that need access, with the value to share. 3) `useContext()` — consumes the value in any child component.

Code Example:

```
const ThemeCtx = createContext();
// Provider wraps the app:
<ThemeCtx.Provider value='dark'>
  <App />
</ThemeCtx.Provider>
// Consumer in any child:
const theme = useContext(ThemeCtx);
```

▢ **Interview Tip:** Common use cases: theme (dark/light), user authentication, language/locale. Mention at least 2 in interviews.

Q44. When should you use Context API vs Props? ▢ PRACTICAL

Short Answer: Use props for data specific to a component. Use Context for truly global data needed by many components at different levels.

Detailed Explanation: Context is not a replacement for all props — it's for data that is 'global' to the app: current user, theme, language settings. Overusing Context causes unnecessary re-renders (every context consumer re-renders when context changes). For complex state, prefer Zustand or Redux over Context.

Practical Example: User login status → Context. A component's specific button color → Props.

▢ **Interview Tip:** Showing awareness of Context's performance limitations (re-rendering all consumers) is an advanced point that impresses interviewers.

PHASE 15–16 — API Handling & Loading/Error States

Q45. How do you fetch API data in a React component? ▢ PRACTICAL

Short Answer: Use the fetch API or axios inside a `useEffect` hook, store the response in state, and render the data from state.

Detailed Explanation: The standard pattern: 1) `useEffect` with `[]` runs after mount. 2) Call the API inside it using fetch/axios. 3) Store result in state with `useState`. 4) Render state data. Always handle loading and error states for a professional UX.

Code Example:

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
```

```
useEffect(() => {
  fetch('https://api.example.com/users')
    .then(res => res.json())
    .then(json => { setData(json); setLoading(false); });
}, []);
```

□ **Interview Tip:** Always mention loading and error states — fetching data without them is incomplete and shows lack of production experience.

Q46. Why is it important to handle loading and error states in API calls? □ PRACTICAL

Short Answer: Without loading/error states, users see a blank screen during fetch and have no feedback when something goes wrong.

Detailed Explanation: A professional UI always shows: 1) A loading spinner or skeleton while data is being fetched. 2) An error message if the API call fails (network error, 404, 500). 3) The actual data once loaded. Without these, the app feels broken and unpolished. This is also essential for accessibility — users need feedback about what's happening.

Code Example:

```
if (loading) return <Spinner />;
if (error) return <ErrorMessage msg={error} />;
return <DataDisplay data={data} />;
```

□ **Interview Tip:** Mention 3 states: loading, error, success (data). This is the standard API pattern in every professional React application.

PHASE 17–21 — React Routing

Q47. What is React Router and why is it needed? □ BASIC

Short Answer: React Router is a third-party library that enables client-side routing in React SPAs, allowing navigation between views without page reloads.

Detailed Explanation: React apps are SPAs with a single HTML file. React Router maps URL paths to specific components, simulating multi-page navigation. Without it, all content would live on a single URL. Key components: BrowserRouter (provides routing context), Routes/Route (defines URL-component mapping), Link/NavLink (navigation without reload).

Code Example:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>
```

□ **Interview Tip:** Always mention the difference between BrowserRouter (uses real URLs) and HashRouter (uses # in URL for compatibility).

Q48. What is the difference between Link and NavLink? □ BASIC

Short Answer: Link navigates without reload. NavLink does the same but also adds an 'active' CSS class to the current route's link automatically.

Detailed Explanation: Link is the basic navigation component — it renders an <a> tag but prevents full page reload. NavLink is an extension of Link that adds an 'active' class (or custom class) to the link when its href matches the current URL. This is used for navigation menus where you want to highlight the current page.

Code Example:

```
<NavLink to='/home' className={({isActive}) => isActive ? 'active' : ''}>Home</NavLink>
```

□ **Interview Tip:** NavLink is preferred for navigation menus. It automatically handles the active state — no extra logic needed.

Q49. What are Dynamic Routes and useParams? □ PRACTICAL

Short Answer: Dynamic routes use a URL parameter (like :id) to render different content based on the URL. useParams() extracts these parameters.

Detailed Explanation: When you have pages like /products/1, /products/2, etc., you define one route with a parameter: <Route path='/products/:id'>. The :id is a URL parameter. Inside the component, useParams() returns an object with the parameter value. This enables data-driven pages like product detail, user profile, blog post pages.

Code Example:

```
// Route definition:
<Route path='/products/:id' element={<ProductDetail />} />
// Inside ProductDetail:
const { id } = useParams();
useEffect(() => fetchProduct(id), [id]);
```

□ **Interview Tip:** Practice this pattern — dynamic routes are essential for any e-commerce or content-driven app.

Q50. What is useNavigate() and how does it differ from Link? □ PRACTICAL

Short Answer: useNavigate() is a hook for programmatic navigation — navigating in response to logic (like after login), not from a click on a link element.

Detailed Explanation: Link renders a clickable anchor for user-initiated navigation. useNavigate() is used when navigation should happen based on code logic: after form submission, after API success, or based on conditions. It returns a navigate function you call with a path string.

Code Example:

```
const navigate = useNavigate();
function handleLogin() {
  // After successful auth:
  navigate('/dashboard');
}
```

□ **Interview Tip:** Use case: redirect after login, redirect after form submit. These are the scenarios to mention.

Q51. What is a Protected Route and how do you implement it? □ PRACTICAL

Short Answer: A Protected Route is a wrapper component that checks if the user is authenticated. If not, it redirects to the login page.

Detailed Explanation: Without route protection, any user can navigate to /dashboard or /admin by typing the URL directly. A ProtectedRoute component checks auth state (from context, state, or localStorage) and either renders the child component (if authenticated) or redirects to /login using the Navigate component.

Code Example:

```
function ProtectedRoute({ children }) {
  const { user } = useAuth();
```

```

    return user ? children : <Navigate to='/login' />;
  }
  // Usage:
  <Route path='/dashboard' element={
    <ProtectedRoute><Dashboard /></ProtectedRoute>
  } />

```

□ **Interview Tip:** This is asked in almost every React interview for apps with auth. Practice writing this component from memory.

Q52. What are Nested Routes and what is the Outlet component? □ PRACTICAL

Short Answer: Nested routes are child routes inside a parent route. The Outlet component renders the matched child route inside the parent's layout.

Detailed Explanation: Nested routing is used for layouts with shared UI (dashboard with sidebar, admin panel). The parent route renders the common layout. Outlet is placed where the child route's component should appear. When navigating to a child route, React Router renders the parent layout with the child component injected at the Outlet position.

Code Example:

```

// Parent route:
<Route path='/dashboard' element={<DashboardLayout />}>
  <Route path='stats' element={<Stats />} />
</Route>
// DashboardLayout.jsx:
return <div><Sidebar /><Outlet /></div>;

```

□ **Interview Tip:** Think of Outlet as a 'slot' in the parent layout. Explain it with the analogy of a frame (parent) and a picture (child).

PHASE 22 — Local Storage

Q53. What is localStorage in JavaScript and how is it used in React? □ BASIC

Short Answer: localStorage is a browser API to store key-value string data persistently (survives page refresh and browser close). Used in React to persist state.

Detailed Explanation: localStorage stores strings only. Use JSON.stringify to store objects and JSON.parse to retrieve them. In React, you typically read from localStorage during initial state setup and write to it inside useEffect when state changes. Common uses: dark mode preference, auth token, shopping cart.

Code Example:

```

// Save:
localStorage.setItem('theme', JSON.stringify({ mode: 'dark' }));
// Read:
const saved = JSON.parse(localStorage.getItem('theme'));
// Remove:
localStorage.removeItem('theme');

```

□ **Interview Tip:** Always mention JSON.stringify and JSON.parse — localStorage only stores strings, and missing this is a common beginner bug.

PHASE 23–25 — Performance Optimization

Q54. What is React.memo and how does it optimise performance? ▢ PRACTICAL

Short Answer: React.memo is a Higher Order Component that prevents a child component from re-rendering if its props haven't changed.

Detailed Explanation: By default, when a parent re-renders, all its children re-render too, even if their props are identical. React.memo wraps a component and memoizes it — React skips re-rendering the child if it receives the same props. Use it for expensive components that receive stable props. Don't over-use it — the memoisation itself has a small cost.

Code Example:

```
const ExpensiveList = React.memo(function List({ items }) {  
  return items.map(i =><Item key={i.id} {...i} />);  
});
```

▢ **Interview Tip:** Key phrase: 'React.memo does a shallow comparison of props.' If props are objects, use useMemo for the object reference.

Q55. What is useMemo and when should you use it? ▢ PRACTICAL

Short Answer: useMemo memoizes the result of an expensive calculation, recomputing it only when specified dependencies change.

Detailed Explanation: Without useMemo, an expensive calculation (like filtering 10,000 items) runs on every re-render. useMemo caches the result and only recalculates when its dependencies change. Don't use it for every calculation — only for genuinely expensive operations, as over-memoization adds complexity without benefit.

Code Example:

```
const filtered = useMemo(() => {  
  return products.filter(p => p.category === selected);  
}, [products, selected]); // Recalculates only when products or selected changes
```

▢ **Interview Tip:** Mention the rule: 'Only use useMemo when you can measure a performance problem. Premature optimisation is the root of all evil.'

Q56. What is useCallback and how does it differ from useMemo? ▢ PRACTICAL

Short Answer: useCallback memoizes a function reference, preventing it from being recreated on every render. useMemo memoizes a computed value.

Detailed Explanation: In JavaScript, every render creates new function instances. When you pass a handler as a prop to a memoized child, the child still re-renders because it receives a 'new' function every time. useCallback solves this by returning the same function reference as long as dependencies don't change.

Code Example:

```
// Without useCallback, handleDelete is new every render:  
const handleDelete = useCallback((id) => {  
  setItems(prev => prev.filter(item => item.id !== id));  
}, []); // Stable reference - never recreated
```

▢ **Interview Tip:** The mental model: useMemo = memoize a value. useCallback = memoize a function. Both prevent unnecessary work.

PHASE 26 — Code Splitting

Q57. What is Code Splitting and why is it important? □ BASIC

Short Answer: Code splitting breaks a large JavaScript bundle into smaller chunks that are loaded on demand, improving initial page load performance.

Detailed Explanation: When React builds an app, it bundles all JS into one file. For large apps, this bundle can be several MBs, making the initial load slow. Code splitting allows the browser to download only the code needed for the current page. `React.lazy()` and `Suspense` enable component-level code splitting. Route-based splitting is the most common approach.

Practical Example: An admin dashboard imported only when the admin navigates to `/admin`, not bundled with the initial app load for regular users.

□ Interview Tip: Mention Core Web Vitals (LCP, FID) — code splitting directly improves these Google page performance metrics.

Q58. How do `React.lazy()` and `Suspense` work together? □ PRACTICAL

Short Answer: `React.lazy()` dynamically imports a component. `Suspense` wraps it and shows a fallback UI while the component's code is loading.

Detailed Explanation: `React.lazy()` takes a function that calls `dynamic import()` and returns a special lazy component. When React first renders this component, it requests the JS chunk from the server. `Suspense` catches this 'loading' state and renders the fallback (a spinner/skeleton). Once loaded, the actual component renders.

Code Example:

```
const AdminPage = React.lazy(() => import('./AdminPage'));  
// In JSX:  
<Suspense fallback={<Spinner />}>  
  <AdminPage />  
</Suspense>
```

□ Interview Tip: Route-based code splitting is the biggest win: lazy load each page/route. This is the first optimisation to apply in any large React app.

QUICK REVISION CHEATSHEET

JSX Rule: Return one root, close all tags, `className` not `class`, camelCase events.

useState: Returns [value, setter]. Setter triggers re-render. Use functional form for sequential updates.

useEffect: `[]` = once, `[dep]` = on dep change, no array = every render. Return cleanup function.

Props: Read-only. Parent → Child. Destructure in params. Never mutate.

State: Mutable. Internal. Triggers re-render on change. Async updates.

Keys: Unique. Stable. From data ID not index (unless static list).

Controlled Component: Input value = state. `onChange` updates state. Single source of truth.

Context API: createContext → Provider → useContext. For global data (theme, auth, locale).

React.memo: Prevents child re-render if props unchanged. Shallow comparison.

useMemo: Memoize expensive computed value. Returns value.

useCallback: Memoize function reference. Returns function.

useRef: Mutable .current. No re-render on change. DOM access + value storage.

React.lazy + Suspense: Lazy = dynamic import. Suspense = fallback during load.

Protected Route: Check auth state. Render children or <Navigate to='/login' />.

Lifting State Up: Move state to nearest common parent of components that need it.

Best of luck in your interviews! ☐ — College Coders